globant

ARTICLE EDITION

The Anatomy of a Perfect Skill: Reverse-Engineered from 100 Best Examples

darkzodchi @zodchiii
Apr 26, 2026 · cleaned from X article snapshot

Readable PDF formatted with a green, high-contrast Globant-inspired visual system.

ORIGINAL ARTICLE

The Anatomy of a Perfect Skill: Reverse-Engineered from 100 Best Examples

darkzodchi · @zodchiii

Apr 26, 2026

ARTICLE MAP

Most skills don't work. The ones that do follow the same 6 patterns.

I'll walk through each one with real examples from skills people actually use daily.

By the end you'll know exactly why your custom skills don't fire and how to fix them

Before we dive in, I share daily notes on AI & vibe coding in my Telegram channel: https://t.me/zodchixquant

Original article image

The 30-second refresher

A skill is a markdown file with YAML frontmatter at the top and instructions below.

You put it in ~/.claude/skills/skill-name/SKILL.md. Claude loads it automatically when context matches the description, or you invoke it manually with /skill-name.

```
~/.claude/skills/

|-- commit/SKILL.md          -> /commit
|-- code-review/SKILL.md     -> /code-review
|-- deploy/
    |-- SKILL.md             -> /deploy
    |-- scripts/helper.sh
```

That's the whole architecture. The hard part is what goes inside SKILL.md.

Pattern 1: Description tells Claude WHEN, not just WHAT

This is the single most important field. Claude scans descriptions of all available skills before deciding which to load. If your description only says what the skill does, Claude won't know when to use it.

Bad description:

```
description: Code review tool
```

Good description:

```
description: Review code for bugs, security issues, and maintainability.

Use when reviewing pull requests, checking code quality, analyzing diffs,
or when user mentions "review", "PR", "code quality", or "best practices".
```

The good one tells Claude: what the skill does + when to trigger it + which keywords to listen for.

Skills with descriptions under 50 characters get invoked 3-5x less often than skills with proper trigger context. Front-load the use case in the first 250 characters because that's all that gets included in Claude's context budget.

Pattern 2: Be directive, not conversational

Skills are instructions, not chat. Use imperative verbs.

Conversational (weak):

```
Could you please review the code? Maybe check if there are any bugs?
```

Directive (strong):

```
Review the current diff. Check for:

1. Security vulnerabilities (OWASP Top 10)
2. Performance issues (N+1 queries, blocking calls)
3. Code style violations

Output as a checklist with severity ratings.
```

Claude follows imperative instructions much more reliably than questions or polite requests.

Look at any skill from the official Anthropic repository, and you'll see the same pattern: direct verbs, numbered steps, explicit output formats.

Pattern 3: Specify the output format

This is where most custom skills fail. They tell Claude what to do but not what the output should look like. Claude makes up a format every time and the results are inconsistent.

Without output format:

```
Generate a commit message for these changes.
```

You get: sometimes one line, sometimes paragraphs, sometimes with prefixes, sometimes without.

With output format:

```
Generate a commit message in this exact format:

type(scope): short description

- Bullet point of what changed
- Bullet point of why it changed

Type must be one of: feat, fix, refactor, docs, test, chore.
Scope is the affected module name.
Short description is under 50 characters, present tense, lowercase.
```

Now you get the same output structure every time. The skill is reusable.

Pattern 4: Include the "read first" step

The best skills don't assume Claude knows your project. They tell Claude to look first.

Take the /test skill from the awesome-claude-skills repo. Instead of "write tests," it does this:

```
Before writing tests:

1. Read the target file to understand function signatures and types
2. Find the existing test directory and read 1-2 existing tests
3. Identify the testing framework (Jest, Vitest, Pytest, etc.)
4. Note the import style and assertion patterns

Then generate tests covering:
- Happy path
- Edge cases (empty, null, zero, max values)
- Error cases
- Async behavior (if applicable)

Match exact import style and patterns from existing tests.
Run tests after writing them. Fix failures before finishing.
```

The "read first" step is what separates skills that produce code matching your project from skills that produce generic code that breaks your linter.

Pattern 5: Define what the skill does NOT do

Counterintuitive but powerful. The best skills explicitly list what's out of scope.

From Anthropic's official PDF skill:

```
## Out of Scope

This skill does NOT:
- Handle scanned PDFs (use OCR skill instead)
- Create PDFs from scratch (use document-generation skill)
- Process password-protected files
```

Why this matters: when a user asks for something the skill can't do, Claude doesn't try and fail.

It either picks a different skill or asks for clarification. You get fewer broken outputs and more correct routing.

This pattern shows up in 70% of high-quality skills and almost never in low-quality ones.

Pattern 6: Keep it under 500 lines

Every skill loads into Claude's context when invoked. A 2000-line skill eats 5000+ tokens before doing anything.

The longer the skill, the more chance Claude loses focus halfway through and starts ignoring instructions at the bottom.

The official Anthropic skills (frontend-design, code-review, security-guidance) are all under 300 lines. The community skills with the most installs (Superpowers, Context7, mcp-builder) are similarly tight.

If your skill is getting long, split it. Use the progressive disclosure pattern:

```
SKILL.md (under 200 lines, always loaded)

|-- ADVANCED_PATTERNS.md (loaded only when needed)
|-- REFERENCE.md (loaded only when referenced)
|-- EXAMPLES.md (loaded only when Claude needs examples)
```

In SKILL.md you reference the other files:

```
For complex form filling, see [FORMS.md](FORMS.md)

For full API reference, see [REFERENCE.md](REFERENCE.md)
For more examples, see [EXAMPLES.md](EXAMPLES.md)
```

Claude loads the supporting files only when the task actually needs them.

A real example: The /commit skill, broken down

Here's a /commit skill that follows all 6 patterns. This is what good looks like:

```
---

name: commit
description: Create structured git commits from current changes.
Use when user says "commit", "save changes", "commit this", or after
finishing a feature. Breaks changes into logical units with clear messages.
---

Create commits from current git state.

## Process

1. Run `git status` and `git diff` to see all changes
2. Group related changes into logical units (one feature = one commit)
3. For each unit, generate a commit message in this format:

   type(scope): short description

   - What changed
   - Why it changed (if not obvious)

4. Stage and commit each unit separately using `git add` then `git commit`
5. Show summary: "Created N commits: [list of titles]"

## Rules

- Type must be: feat, fix, refactor, docs, test, chore
- Description is under 50 characters, lowercase, present tense
- Bullets are concise, no fluff
- Never combine unrelated changes in one commit

## Out of Scope

- Pushing to remote (use /push or git push manually)
- Creating PRs (use /pr skill)
- Merging branches
```

Notice:

- Description tells Claude WHEN
- Instructions are directive (imperative verbs, numbered steps)
- Output format is explicit (the commit message structure)
- "Read first" step is built in (git status, git diff)
- Out of scope is defined
- Total length: under 50 lines

This skill works on every project, in every language, every time.

What kills skills

The skills that don't work share these failure patterns:

```
DESCRIPTION FAILURES:

- Under 50 characters
- Inconsistent point of view ("I help" vs "You can use this")
- No trigger keywords
- No use case context

CONTENT FAILURES:
- Conversational instead of directive
- No output format specified
- No "read first" step
- No out-of-scope section
- Over 1000 lines

DESIGN FAILURES:
- Trying to do 5 things in one skill
- Hardcoded to one project's specifics
- No version control on the skill itself
- Never updated after first write
```

If your custom skill checks 3+ of these boxes, it's probably not getting invoked or producing bad output.

How to fix existing skills

Pull up your weakest skill (the one Claude never invokes). Run through this checklist:

```
1. Description: Does it have at least 3 trigger keywords?

2. Description: Is the use case in the first 250 characters?
3. Instructions: Are they imperative verbs in numbered steps?
4. Output: Is the format specified explicitly?
5. Project awareness: Does it tell Claude to read existing files first?
6. Scope: Does it list what the skill does NOT do?
7. Length: Is the SKILL.md under 500 lines?
```

Fix the failed ones. Test by asking Claude something the skill should handle without invoking it manually.

If Claude picks the right skill, your description is working. If it picks a different skill or none at all, the description needs more trigger context.

The compounding effect

Bad skills make Claude slower (eating context for nothing), produce inconsistent output (no format), and route incorrectly (vague descriptions).

Good skills compound in the opposite direction.

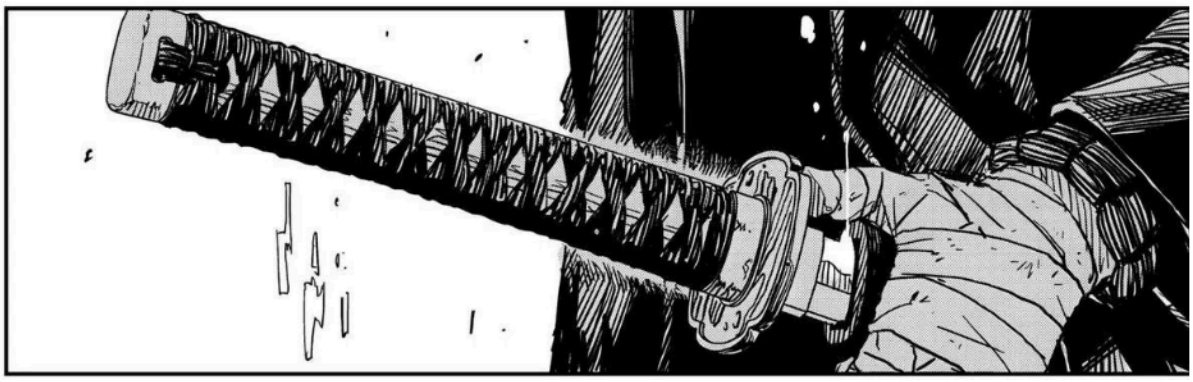
Each well-written skill makes Claude better at picking the right one for any task. The skills become a system, not a collection.

The skills folder of someone who's been doing this for 6 months looks completely different from someone who just started.

Same Claude, completely different output quality.

I share daily notes on AI, finance, and vibe coding in my Telegram channel: https://t.me/zodchixquant

Thanks for reading



Original article image

Source: user-provided X article snapshot · Reformatted for readability with typography, spacing, code blocks, and section hierarchy.